



4. Estructuras de los programas en Python y en C

Profesor: Juan Moreno García

Índice

1. Objetivos.
2. Estructura programas C de Arduino. Blink.
3. Un ejemplo.
4. Programas a realizar.
5. Conclusiones.

1. Objetivos

1. Recordar **estructura de programas Python**.
2. Aprender la **estructura de los programas de NodeMCU**.
3. Recordar **funciones en Python**.
4. Conocer la **estructura de las funciones en C de NodeMCU**.
5. Recordar **entrada/salida en Python**.
6. Aprender **entrada/salida en C de NodeMCU**.

2. Estructura programas C de Arduino. Blink.

```
blink 5
#define LED_BUILTIN 2

void setup() {
    pinMode(LED_BUILTIN, OUTPUT); // LED integrado conectado al pin 2.
}

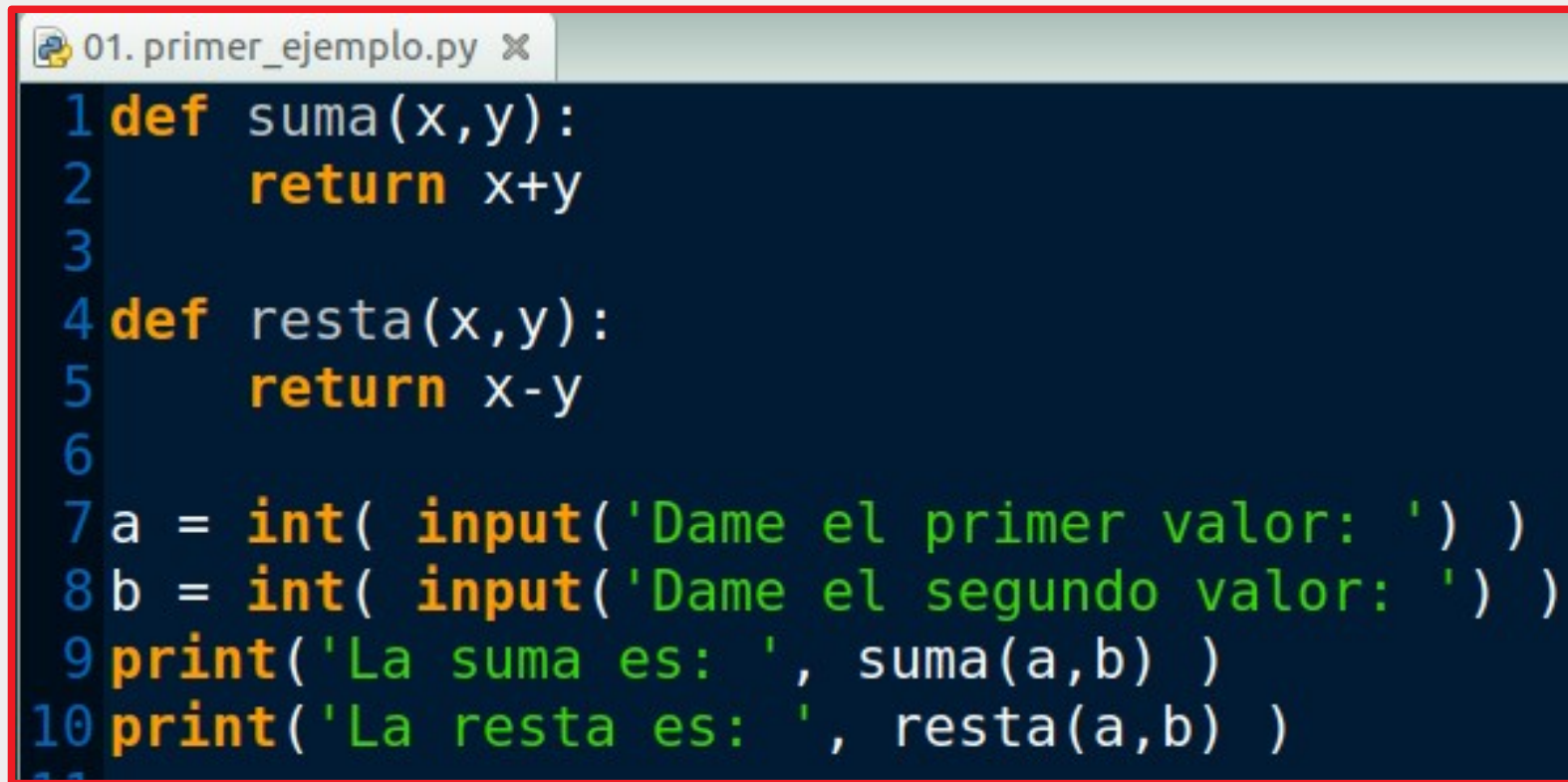
void loop() {
    digitalWrite(LED_BUILTIN, LOW); // LED encendido si está a nivel bajo
    delay(1000); // Esperar un segundo
    digitalWrite(LED_BUILTIN, HIGH); // Apagar el LED a nivel alto
    delay(2500); // Esperar 2,5 segundos
}
```

- * **define**: Crea etiquetas que pueden ser utilizadas en el programa.
- * **setup()**: Se ejecuta 1 vez y se utiliza para inicializar dispositivos y variables globales.
- * **loop()**: Es la función principal. Se ejecuta cada cierto tiempo ejecutando las instrucciones dentro de la misma.

3. Un ejemplo.

Programa que utilice dos funciones, una para sumar dos enteros y otra para restar dos enteros.

Versión Python



```
01. primer_ejemplo.py x
1 def suma(x,y):
2     return x+y
3
4 def resta(x,y):
5     return x-y
6
7 a = int( input('Dame el primer valor: ') )
8 b = int( input('Dame el segundo valor: ') )
9 print('La suma es: ', suma(a,b) )
10 print('La resta es: ', resta(a,b) )
```

3. Un ejemplo.

Programa que utilice dos funciones, una para sumar dos enteros y otra para restar dos enteros. **Versión C de NodeMCU**

```
// Funcion que suma dos enteros, devuelve
// un entero y recibe como parametros dos enteros
int suma(int x, int y) {
    return x+y; // Orden para retornar el valor
}

// Funcion que resta dos enteros, devuelve
// un entero y recibe como parametros dos enteros
int resta(int x, int y) {
    return x-y; // Orden para retornar el valor
}

void setup() {
    Serial.begin(115200);
}
```

```
// Esta función se ejecuta constantemente
void loop() {
    int a, b;
    String r;

    // Leer primer entero
    Serial.println("Dame el primer entero");
    r = leerLinea(); // recibe info hasta '\n'
    a = r.toInt();
    // Leer segundo entero
    Serial.println("Dame el segundo entero");
    r = leerLinea(); // recibe info hasta '\n'
    b = r.toInt();

    // Muestra los valores leídos
    Serial.println("a = " + String( a ) + ". b = " + String(b) );
    // Ejecuta la suma, la convierte a String y muestra el
    // mensaje en el Monitor Serial
    Serial.println("La suma es: " + String( suma(a,b) ) );
    // Ejecuta la resta, la convierte a String y muestra el
    // mensaje en el Monitor Serie
    Serial.println("La resta es: " + String( resta(a,b) ) );
    Serial.println();
}
```

4. Programas a realizar.

Ejercicio:

Hacer dos programas (uno en Python y otro en C) con dos funciones y un principal donde:

- * La función 1 implementa la ecuación $f(x)=2*x+1$.
- * La función 2 implementa la ecuación $f(x,y)=3*x^2+2*y$.
- * El programa principal debe invocar ambas funciones y mostrar por pantalla (Monitor Serie en caso del NodeMCU) el siguiente mensaje:
“a = X. B = Y. Función 1 devuelve R y función 2 devuelve S”

5. Conclusiones

1. Los programas *Python* se componen de funciones y un programa principal que colocaremos fuera de todas las funciones.
2. Los programas C de *NodeMCU* se componen de tres partes:
 - (1) Una parte inicial para incluir las librerías utilizadas y declarar las variables globales.
 - (2) La función *setup()* se utiliza para inicializar las variables y dispositivos conectados al NodeMCU.
 - (3) La función *loop()* se ejecuta cada cierto tiempo y repite las órdenes dentro de la misma. Es un bucle infinito.
3. Las funciones en *Python* se definen utilizando la palabra **def** seguida de su nombre y los parámetros. No se indican tipos de datos ni para el valor de retorno ni para los parámetros. Las tabulaciones indican la función a la que pertenecen las órdenes.
4. Las funciones de C tienen 3 partes: (1) el valor de retorno; (2) los parámetros; y (3) el código dentro del cuerpo de la función (indicado entre llaves). Hay que indicar los tipos de parámetros y del valor de retorno.

5. Conclusiones

- 5. La orden *print()* muestra por pantalla lo que se le pase como parámetro.
- 6. C del *NodeMCU* utiliza el módulo *Serial* para mostrar la información en el *Monitor Serie*.
- 7. La función *leerLinea()* lee una línea hasta el carácter '\n'.